Competitive Programming

Questions and Answers

Question 1 - Find Duplicate in Array

Problem Statement

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format: First Line - Number of elements. n Lines - n Elements.

Output Format: Element x - That is repeated

Example

Input	Output
5 1 1 2 3 4	1

Solution 1 - Nested loops (no extra space)

```
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[i] == a[j]) {
                printf("%d", a[i]);
                return 0;
            }
        }
    }

    return 0;
}
```

Output of the program

Input	Output
5 1 1 2 3 4	1

Solution 2 - Counting array

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int count[n + 1];

    for (int i = 0; i <= n; i++)
        count[i] = 0;

    for (int i = 0; i < n; i++) {
        int x;
        scanf("%d", &x);
        count[x]++;
    }

    for (int i = 1; i <= n; i++) {
        if (count[i] > 1) {
            printf("%d", i);
            break;
        }
    }

    return 0;
}
```

Output of the program

Input	Output
5 1 1 2 3 4	1

Approach

Solution 1: compare every element a[i] with all later elements a[j]. The first pair with a[i] == a[j] gives the repeated number. The array is only read, never changed, and no extra array is used.

Solution 2: since every value is between 1 and n, use count[value] as a frequency table. Read all elements and do count[x]++. Then scan values 1 to n and print the first value whose count is greater than 1.

Time Complexity

Time complexity: Solution 1: O(n^2) time, O(1) space | Solution 2: O(n) time, O(n) space
Solution 1 needs two nested loops over the array. Solution 2 needs one pass to count and one pass to search, but it uses an extra count[] array of size n + 1.

Question 2 - Intersection of Two Sorted Arrays

Problem Statement

Find the intersection of two sorted arrays.

OR in other words, Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format: The first line contains T, the number of test cases. Following T lines contain: (1) Line 1 contains N1, followed by N1 integers of the first array. (2) Line 2 contains N2, followed by N2 integers of the second array.

Output Format: The intersection of the arrays in a single line

Example 1: Input: 1 / 3 10 17 57 / 6 2 7 10 15 57 246 Output: 10 57

Example 2: Input: 1 / 6 1 2 3 4 5 6 / 2 1 6 Output: 1 6

Example

Input	Output
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer (C program)

```
#include <stdio.h>

int main() {
    int T;
    scanf("%d", &T);

    while (T--) {
        int n1, n2;
        scanf("%d", &n1);

        int a[n1];
        for (int i = 0; i < n1; i++)
            scanf("%d", &a[i]);

        scanf("%d", &n2);

        int b[n2];
        for (int i = 0; i < n2; i++)
            scanf("%d", &b[i]);

        int i = 0, j = 0;

        while (i < n1 && j < n2) {
            if (a[i] == b[j]) {
                printf("%d ", a[i]);
                i++;
                j++;
            }
            else if (a[i] < b[j]) {
                i++;
            }
            else {
                j++;
```

```
            }
        }

        printf("\n");
    }

    return 0;
}
```

Output of the program

Input	Output
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Approach

1. Use two pointers: i on the first array and j on the second array, both starting at 0.
2. If a[i] == b[j], the element is in both arrays: print it and move both pointers.
3. If a[i] < b[j], move i forward. Otherwise move j forward (both arrays are sorted, so the smaller value cannot match anything later).
4. Stop when either array is finished, then print a new line for the next test case.

Time Complexity

Time complexity: O(N1 + N2) per test case
Each pointer only moves forward, so at most N1 + N2 steps are made. No extra space is needed apart from the input arrays.

Question 3 - Pair with Difference k

Problem Statement

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format: First Line n - Number of elements in an array. Next n Lines - N elements in the array. k - Non - Negative Integer.

Output Format: 1 - If pair exists. 0 - If no pair exists.

Explanation for the given Sample Testcase: YES as 5 - 1 = 4. So Return 1.

Example

Input	Output
3 1 3 5 4	1

Answer (C program)

```
#include <stdio.h>

int main() {
    int n, k;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &k);

    int i = 0, j = 1;

    while (i < n && j < n) {
        if (i == j) {
            j++;
        }
        else if (a[j] - a[i] == k) {
            printf("1");
            return 0;
        }
        else if (a[j] - a[i] < k) {
            j++;
        }
        else {
            i++;
        }
    }

    printf("0");

    return 0;
}
```

Output of the program

Input	Output
3 1 3 5 4	1

Approach

1. Use two pointers i = 0 and j = 1 on the sorted array.
2. If i == j, move j forward so the two indices are different.
3. If a[j] - a[i] == k, a pair exists: print 1.
4. If the difference is smaller than k, move j forward to make it bigger. If it is bigger than k, move i forward to make it smaller.
5. If a pointer reaches the end without a match, print 0.

Time Complexity

Time complexity: O(n)
Both pointers only move forward, so there are at most 2n steps. A brute force check of every pair would be $O(n^2)$.